Vorlesung: Datenbanken und Informationssysteme

Thema: Datenbank-Zugriffe (Datenbankprogrammierung)

Inhalt:

1. **Einführung**
2. **Grundlagen der Datenbank-Zugriffe**
3. **Datenbank-Zugriffe mit SQL**
4. **Datenbank-Zugriffe in der Programmierung**
5. **Weitere Datenzugriffsarten**
6. **Zusammenfassung**

1. Einführung

Datenbanken sind ein wesentlicher Bestandteil moderner Anwendungen. Sie ermöglichen das effiziente Speichern, Abrufen und Verwalten großer Datenmengen. Bei der Entwicklung von Datenbankanwendungen ist der Zugriff auf die Datenbank von entscheidender Bedeutung. Diese Ausarbeitung befasst sich mit den Grundlagen der Datenbank-Zugriffe und deren Umsetzung sowohl mit SQL, der Programmierung als auch mit weiteren Datenbank-Zugriffs-Mechanismen.

2. Grundlagen der Datenbank-Zugriffe

2.1 **Datenbanksysteme**

Database Management Systeme (DBMS) sind Softwareanwendungen, um die Verwaltung von Datenbanken zu erleichtern. Sie bieten Mechanismen zur Organisation, Speicherung, Abfrage und Aktualisierung von Daten. Es gibt verschiedene Arten von Datenbanksystemen, die je nach ihren Datenmodellen und Funktionalitäten kategorisiert werden können. Hier sind ein paar gängige Arten von Datenbanksystemen:

- Relationale Datenbanksysteme (RDBMS): Relationale Datenbanksysteme sind die am weitesten verbreitete Art von Datenbanksystemen. Sie basieren auf dem relationalen Datenmodell, das Daten in tabellarischer Form organisiert. In einem RDBMS werden Daten in Relationen (Tabellen) gespeichert und Beziehungen zwischen den Tabellen durch Schlüssel hergestellt. Beispiele für relationale Datenbanksysteme sind MySQL, Oracle Database und Microsoft SQL Server.

- Hierarchische Datenbanksysteme: Hierarchische Datenbanksysteme organisieren Daten in einer Baumstruktur, in der jede Entität eine übergeordnete und mehrere untergeordnete Entitäten haben kann. Diese Art von Datenbanksystemen eignet sich gut für Anwendungen mit klar definierten Hierarchien, wie beispielsweise Verzeichnisse oder Organisationsstrukturen. Ein bekanntes hierarchisches Datenbanksystem ist IBM's IMS (Information Management System).

- Objektorientierte Datenbanksysteme (OODBMS): Objektorientierte Datenbanksysteme erweitern das relationale Modell, indem sie die Konzepte der Objektorientierung einführen. Sie ermöglichen die Speicherung komplexer Objekte mit Attributen und Methoden in der Datenbank. Dies erleichtert die Modellierung von Daten, die objektorientierte Konzepte verwenden, wie zum Beispiel in objektorientierten Programmiersprachen. Beispiele für objektorientierte Datenbanksysteme sind MongoDB und db4o.

- NoSQL-Datenbanksysteme: NoSQL (Not Only SQL) ist eine Sammelbezeichnung für Datenbanksysteme, die keine relationale Struktur verwenden. Diese Systeme sind flexibler als relationale Datenbanksysteme und eignen sich gut für den Umgang mit unstrukturierten oder semi-strukturierten Daten, wie sie in Big-Data-Anwendungen, sozialen Netzwerken oder Echtzeit-Analysen vorkommen. Beispiele für NoSQL-Datenbanksysteme sind MongoDB, Cassandra und Redis.

Relationale Datenbanken und NoSQL-Datenbanken sind zwei verschiedene Ansätze zur Datenorganisation und -verwaltung in Datenbanksystemen. Die Unterschiede dabei sind folgende:

Relationale Datenbanken:

- Struktur: Relationale Datenbanken verwenden das relationale Datenmodell, bei dem Daten in Tabellen organisiert werden. Tabellen haben Spalten (Attribute) und Zeilen (Datensätze).

- Schema: Relationale Datenbanken erfordern ein vordefiniertes Schema, das die Struktur der Tabellen, die Typen der Attribute und die Beziehungen zwischen den Tabellen beschreibt.

- Datenintegrität: Relationale Datenbanken legen Wert auf Datenintegrität und erzwingen Regeln wie Primärschlüssel, Fremdschlüssel und eindeutige Werte.

- Flexibilität: Relationale Datenbanken sind weniger flexibel, wenn es um die Speicherung unstrukturierter oder stark variabler Daten geht. Änderungen am Schema können mühsam sein.

- Skalierbarkeit: Relationale Datenbanken können vertikal (durch Hinzufügen von Ressourcen zu einem einzelnen Server) und horizontal (durch Hinzufügen weiterer Server) skaliert werden.

- Transaktionen: Relationale Datenbanken unterstützen ACID-Transaktionen (Atomicity, Consistency, Isolation, Durability) und gewährleisten so Datenkonsistenz.

NoSQL-Datenbanken (Not Only SQL):

- Datenmodell: NoSQL-Datenbanken verwenden unterschiedliche Datenmodelle, wie z.B. Dokumente, Schlüssel-Wert-Paare, Spaltenfamilien oder Graphen. Dadurch können sie unstrukturierte oder stark variierende Daten effizienter speichern.

- Flexibilität: NoSQL-Datenbanken sind flexibler in Bezug auf das Datenmodell und erlauben das Hinzufügen oder Ändern von Attributen in den Datensätzen ohne strikte Schemaanforderungen.

- Skalierbarkeit: NoSQL-Datenbanken sind für horizontale Skalierung optimiert. Sie können Daten auf mehrere Server verteilen und dadurch höhere Lasten und größere Datenmengen verarbeiten.

- Abstriche bei Konsistenz: NoSQL-Datenbanken bieten oft weniger strenge Konsistenzgarantien als relationale Datenbanken. Stattdessen setzen sie auf hohe Verfügbarkeit und Partitionstoleranz (CAP-Theorem).

- Performance: Durch den Verzicht auf komplexe Relationen und Joins können NoSQL-Datenbanken in bestimmten Szenarien eine höhere Performance erreichen.

- Anwendungsbereich: NoSQL-Datenbanken sind besonders gut geeignet für Big-Data-Anwendungen, Echtzeit-Analysen, Content-Management-Systeme und Szenarien, in denen eine hohe Skalierbarkeit und Flexibilität gefordert sind.

2.2 Datenbankzugriffsmodelle

CRUD-Operationen, auch als Create, Read, Update, Delete bezeichnet, sind grundlegende Operationen zur Datenmanipulation in vielen Datenbanksystemen. Sie ermöglichen das Erstellen, Lesen, Aktualisieren und Löschen von Daten in einer Datenbank. Hier ist eine Erläuterung der einzelnen CRUD-Operationen:

1. Create (Erstellen): Die Create-Operation wird verwendet, um neue Daten in der Datenbank zu erstellen. Dabei werden neue Datensätze oder Dokumente mit den erforderlichen Attributen und Werten hinzugefügt. Die Create-Operation kann verschiedene Formen annehmen, je nach dem Datenbanksystem und der verwendeten Programmiersprache. In der Regel wird eine INSERT- oder CREATE-Methode verwendet, um neue Daten in die Datenbank einzufügen.

2. Read (Lesen): Die Read-Operation dient dazu, Daten aus der Datenbank abzurufen. Sie ermöglicht es, gespeicherte Informationen zu lesen und anzuzeigen. Die Read-Operation kann als SELECT-Abfrage in SQL oder als Find-Operation in NoSQL-Datenbanken formuliert werden. Sie ermöglicht es, nach bestimmten Kriterien zu suchen, eine bestimmte Anzahl von Datensätzen abzurufen oder die gesamte Datenbank abzufragen.

3. Update (Aktualisieren): Die Update-Operation wird verwendet, um vorhandene Daten in der Datenbank zu ändern oder zu aktualisieren. Mit dieser Operation können bestimmte Attribute oder Werte in einem Datensatz geändert werden, ohne den gesamten Datensatz zu löschen und neu zu erstellen. Die Update-Operation erfolgt normalerweise durch die Verwendung von UPDATE-Anweisungen in SQL oder entsprechenden Methoden in der Programmierung.

4. Delete (Löschen): Die Delete-Operation ermöglicht das Entfernen von Daten aus der Datenbank. Sie kann verwendet werden, um einzelne Datensätze oder ganze Datensatzgruppen zu löschen. Die Delete-Operation wird durch die Verwendung von DELETE-Anweisungen in SQL oder entsprechenden Methoden in der Programmierung durchgeführt.

CRUD-Operationen sind grundlegend für die Interaktion mit Datenbanken und bilden die Basis für die meisten Datenbankanwendungen. Sie ermöglichen das Hinzufügen neuer Daten, das Abrufen von Informationen, das Aktualisieren von Datensätzen und das Löschen von Daten, um die Datenbank konsistent und aktuell zu halten.

Transaktionen sind ein zentrales Konzept in der Datenbankverwaltung und dienen dazu, eine Gruppe von Datenbankoperationen als eine logische Einheit zu behandeln. Sie stellen sicher, dass diese Operationen entweder vollständig oder gar nicht ausgeführt werden. Eine Transaktion ermöglicht es, mehrere Schritte der Datenmanipulation zu einer kohärenten Einheit zusammenzufassen.

Um die Integrität und Zuverlässigkeit von Transaktionen zu gewährleisten, wurden die ACID-Eigenschaften eingeführt:

1. Atomicity (Atomarität): Die Atomicity-Eigenschaft bedeutet, dass eine Transaktion als Ganzes betrachtet wird und entweder vollständig oder gar nicht ausgeführt wird. Es gibt keine Zwischenzustände, in denen einige Teile der Transaktion angewendet und andere nicht angewendet wurden. Wenn ein Schritt innerhalb einer Transaktion fehlschlägt, werden alle vorherigen Schritte, die bereits ausgeführt wurden, rückgängig gemacht (ROLLBACK). Dadurch wird die Datenbank in ihren vorherigen konsistenten Zustand zurückgesetzt.

2. Consistency (Konsistenz): Die Consistency-Eigenschaft stellt sicher, dass eine Transaktion die Konsistenzregeln der Datenbank wahrt. Diese Regeln können z. B. die Beibehaltung von Referenzintegrität oder anderen Geschäftsregeln beinhalten. Nach Abschluss einer Transaktion sollte sich die Datenbank in einem gültigen und konsistenten Zustand befinden.

3. Isolation (Isolation): Die Isolation-Eigenschaft stellt sicher, dass mehrere gleichzeitige Transaktionen unabhängig voneinander ausgeführt werden, als ob sie seriell ablaufen würden. Sie schützt Transaktionen vor gegenseitigen Beeinflussungen oder Inkonsistenzen, die auftreten könnten, wenn mehrere Transaktionen gleichzeitig auf die Datenbank zugreifen. Isolation verhindert, dass Daten inkonsistent oder ungenau werden, wenn sie von verschiedenen Transaktionen gleichzeitig gelesen oder geschrieben werden.

4. Durability (Dauerhaftigkeit): Die Durability-Eigenschaft garantiert, dass die Auswirkungen einer abgeschlossenen Transaktion dauerhaft in der Datenbank gespeichert werden. Nachdem eine Transaktion erfolgreich abgeschlossen wurde und das COMMIT ausgeführt wurde, sollten die Änderungen permanent in der Datenbank gespeichert bleiben, selbst bei einem Systemfehler, Stromausfall oder einem anderen unerwarteten Ereignis. Die Daten werden in der Regel durch Protokollierung und regelmäßige Sicherungen geschützt, um ihre Dauerhaftigkeit sicherzustellen.

3. Datenbank-Zugriffe mit SQL

3.1 Structured Query Language (SQL)

SQL (Structured Query Language) ist eine speziell entwickelte Sprache für die Kommunikation mit relationalen Datenbankmanagementsystemen (RDBMS). Mit SQL können Sie Datenbanken erstellen, Tabellen definieren, Daten abrufen, Daten hinzufügen, aktualisieren und löschen sowie komplexe Abfragen durchführen. Hier sind einige grundlegende Syntax- und Grundlagenkonzepte von SQL:

SQL bietet verschiedene Operatoren für Bedingungen, um Datensätze zu filtern, z. B.:

- Vergleichsoperatoren (=, <, >, <=, >=, <>)
- Logische Operatoren (AND, OR, NOT)
- LIKE-Operator für Musterabgleiche (z. B. WHERE Name LIKE 'M%')
- IN-Operator für den Vergleich mit einer Liste von Werten (z. B. WHERE Alter IN (25, 30, 35))

Erstellen einer Tabelle: Um eine Tabelle in einer Datenbank zu erstellen, verwenden Sie die CREATE TABLE-Anweisung. Sie müssen den Tabellennamen und die Spaltennamen sowie den Datentyp und eventuelle Einschränkungen definieren.

```
CREATE TABLE Kunden (
  ID INT PRIMARY KEY,
  Name VARCHAR(50),
  Alter INT,
  Email VARCHAR(100)
);
```

Erstellung einer Tabelle Kunde mit den Attributen ID, Name, Alter und Email

Einfügen von Daten: Um Daten in eine Tabelle einzufügen, verwenden Sie die INSERT INTO-Anweisung. Sie müssen angeben, in welche Tabelle und welche Spalten die Daten eingefügt werden sollen.

```
INSERT INTO Kunden (ID, Name, Alter, Email)
VALUES (1, 'Max Mustermann', 30, 'max@example.com');
```

Einfügen in die Tabelle Kunde mit den Daten (in Values)

Aktualisieren von Daten: Um Daten in einer Tabelle zu aktualisieren, verwenden Sie die UPDATE-Anweisung. Sie müssen angeben, welche Tabelle aktualisiert werden soll und welche Spaltenwerte geändert werden sollen. Sie können auch Bedingungen angeben, um bestimmte Datensätze zu aktualisieren.

```
UPDATE Kunden
SET Alter = 35
WHERE ID = 1;
```

Updaten des Alters auf 35 für den Kunden mit der ID 1

Löschen von Daten: Um Daten aus einer Tabelle zu löschen, verwenden Sie die DELETE FROM-Anweisung. Sie müssen angeben, aus welcher Tabelle Datensätze gelöscht werden sollen. Sie können auch Bedingungen angeben, um bestimmte Datensätze zu löschen.

```
DELETE FROM Kunden
WHERE ID = 1;
```

Löschen des Kunden mit der ID 1

Abfragen von Daten: Um Daten aus einer Tabelle abzurufen, verwenden Sie die SELECT-Anweisung. Sie können angeben, welche Spalten abgerufen werden sollen und optional Bedingungen angeben, um bestimmte Datensätze zu filtern.

```
SELECT Name, Alter
FROM Kunden
WHERE Alter > 25;
```

Abruf des Namens und Alters der Kunden, die älter als 25 Jahre sind

Dies sind nur grundlegende Konzepte und Syntaxbeispiele von SQL. SQL bietet jedoch eine umfangreiche Palette von Funktionen, um komplexe Abfragen, Aggregationen, Joins und andere Datenmanipulationen durchzuführen. Es gibt auch fortgeschrittenere Konzepte wie Indizes, Ansichten und gespeicherte Prozeduren, die in SQL implementiert werden

3.2 Datenbankabfragen

Filtern, Sortieren und Aggregieren von Daten mit SQL ermöglicht es, gezielte Abfragen auf Datenbanken durchzuführen. Mit Filtern können bestimmte Datensätze basierend auf definierten Kriterien ausgewählt werden. Das Sortieren ermöglicht die Anordnung der Ergebnisse nach bestimmten Spaltenwerten. Aggregationen ermöglichen die Berechnung von Zusammenfassungen wie Summen, Durchschnitten oder maximalen/ minimalen Werten über eine Gruppe von Datensätzen. Diese Funktionen bieten Flexibilität und Kontrolle über die Darstellung und Manipulation von Daten in SQL-Abfragen.

Die Verwendung von Joins und Unterabfragen in SQL ermöglicht es, Daten aus mehreren Tabellen zusammenzuführen und komplexe Abfragen zu erstellen. Joins verbinden Tabellen basierend auf gemeinsamen Spaltenwerten, um zusammengehörige Daten abzurufen. Dadurch können Informationen aus verschiedenen Tabellen kombiniert werden, um umfassendere Ergebnisse zu erzielen. Unterabfragen sind Abfragen, die innerhalb einer Hauptabfrage eingebettet sind und als temporäre Tabellen oder Datensätze dienen. Sie ermöglichen das Filtern, Sortieren oder Aggregieren von Daten basierend auf Ergebnissen einer anderen Abfrage. Die Verwendung von Joins und Unterabfragen erweitert die Möglichkeiten von SQL und ermöglicht komplexe Abfragen über mehrere Datenquellen hinweg.

4. Datenbank-Zugriffe in der Programmierung

4.1 Programmiersprachen

Während SQL als Standardabfragesprache für relationale Datenbanken fungiert, bieten moderne Programmiersprachen oft spezifische Funktionen, Bibliotheken oder Frameworks, um den Zugriff auf Datenbanken zu erleichtern. Dadurch haben Entwickler die Freiheit, die Programmiersprache ihrer Wahl zu verwenden und gleichzeitig effizient mit Datenbanken zu interagieren. Die Vielfalt an Möglichkeiten ermöglicht es, den Datenbankzugriff an die spezifischen Anforderungen eines Projekts anzupassen und dabei die Stärken der gewählten Programmiersprache zu nutzen.

- Java mit dem JDBC-Framework für den Datenbankzugriff.

- Python mit dem SQLAlchemy-Framework für die Interaktion mit relationalen Datenbanken.

- C# mit ADO.NET für den Datenbankzugriff im .NET-Framework.

- PHP mit dem MySQLi-Modul für den Zugriff auf MySQL-Datenbanken.

In der heutigen Zeit ist nahezu jede gängige Programmiersprache für den Datenbankzugriff geeignet. Während SQL als die Standardabfragesprache für relationale Datenbanken dient, bieten viele Programmiersprachen spezifische Bibliotheken, Frameworks oder APIs an, um den Datenbankzugriff zu erleichtern. Dadurch haben Entwickler die Flexibilität, die Programmiersprache ihrer Wahl zu verwenden und gleichzeitig effektiv mit Datenbanken zu interagieren. Letztendlich hängt die Entscheidung, welche Programmiersprache verwendet wird, vom Projektumfang, den Anforderungen und den persönlichen Präferenzen ab.

4.2 Datenbankanbindung

Im Folgenden betrachten wir den wichtigen Schritt des Verbindungsaufbaus zur Datenbank, um eine effektive Interaktion zu ermöglichen. Dabei sind einige grundlegende Schritte zu beachten, die im Kontext einer Hochschulvorlesung von Bedeutung sind:

Zunächst sollten Sie den passenden Datenbanktreiber für Ihre gewählte Programmiersprache auswählen. Dieser Treiber fungiert als Schnittstelle zwischen Ihrer Anwendung und der Datenbank. Es ist wichtig sicherzustellen, dass der Treiber mit dem verwendeten Datenbankmanagementsystem kompatibel ist.

Anschließend importieren Sie die erforderlichen Bibliotheken oder Module in Ihren Programmcode, um auf die Funktionen und Methoden für die Datenbankverbindung zugreifen zu können. Abhängig von Ihrer Programmiersprache gibt es spezifische Bibliotheken, die für den Datenbankzugriff entwickelt wurden.

Um eine erfolgreiche Verbindung herzustellen, müssen Sie die relevanten Verbindungsinformationen bereitstellen. Dazu gehören unter anderem der Hostname oder die IP-Adresse des Datenbankservers, der Port, der Name der Datenbank sowie die Anmeldeinformationen wie Benutzername und Passwort. Diese Informationen sind entscheidend für den erfolgreichen Verbindungsaufbau.

Mit den bereitgestellten Verbindungsinformationen können Sie nun die Verbindung zur Datenbank herstellen. Verwenden Sie die entsprechenden Funktionen oder Methoden des Datenbanktreibers, um die Verbindung herzustellen. Achten Sie dabei darauf, mögliche Fehler oder Ausnahmen abzufangen, um eine zuverlässige Fehlerbehandlung zu gewährleisten.

Nachdem die Verbindung hergestellt wurde, ist es ratsam, diese auf ihre erfolgreiche Einrichtung zu überprüfen. Führen Sie eine einfache Testabfrage oder -operation aus, um sicherzustellen, dass die Verbindung ordnungsgemäß funktioniert. Dadurch können potenzielle Probleme frühzeitig erkannt und behoben werden.

Nach Abschluss der gewünschten Datenbankoperationen sollten Sie die Verbindung ordnungsgemäß schließen, um Ressourcen freizugeben. Verwenden Sie die entsprechende Funktion oder Methode des Datenbanktreibers, um die Verbindung zu beenden.

Es ist wichtig, auch Sicherheitsaspekte bei der Verbindungsherstellung zur Datenbank zu berücksichtigen. Speichern Sie beispielsweise Passwörter niemals im Klartext und nutzen Sie verschlüsselte Verbindungen, um die Sicherheit Ihrer Daten zu gewährleisten.

Mit diesen Schritten sind Sie in der Lage, eine stabile Verbindung zur Datenbank herzustellen und Datenbankzugriffe in Ihrer Anwendung zu ermöglichen. Beachten Sie jedoch, dass die genaue Vorgehensweise und Syntax je nach verwendeter Programmiersprache und Datenbankmanagementsystem variieren können.

4.3 Ausführung von SQL-Abfragen

Bei der Ausführung von SQL-Abfragen ist es wichtig, sicherzustellen, dass die Datenbankabfragen effizient und sicher sind. Zwei wichtige Konzepte, die dabei eine Rolle spielen, sind "Prepared Statements" und "Parameterbindung".

Prepared Statements ermöglichen die Trennung der Abfrage und der dazugehörigen Daten. Anstatt die Abfrage direkt mit den Werten zu interpolieren, wird eine Vorlage für die Abfrage erstellt, in der Platzhalter für die Parameter verwendet werden. Diese Platzhalter werden dann zur Laufzeit mit den entsprechenden Werten verbunden.

Die Parameterbindung bietet mehrere Vorteile. Erstens verbessert sie die Leistung, da die Datenbank die Abfrage einmal vorbereiten kann und sie dann bei Bedarf mit unterschiedlichen Werten ausführt, ohne den gesamten Vorbereitungsprozess wiederholen zu müssen. Zweitens erhöht sie die Sicherheit, da sie Schutz vor SQL-Injektionsangriffen bietet. Durch die Verwendung von parametrisierten Abfragen werden die eingegebenen Werte automatisch als Daten interpretiert und nicht als Teil der Abfrage selbst.

Ein Beispiel für die Verwendung von Prepared Statements und Parameterbindung in SQL könnte wie folgt aussehen (verwendet wird die Programmiersprache Java mit der JDBC-Schnittstelle):

```
String sql = "SELECT * FROM customers WHERE age > ?";
PreparedStatement statement = connection.prepareStatement(sql);
statement.setInt(1, 18); // Setzen des Wertes für den Platzhalter
ResultSet resultSet = statement.executeQuery();
// Verarbeitung des Abfrageergebnisses
```

Verwendung eines Prepared Statements in Java (JDBC)

In diesem Beispiel wird eine SQL-Abfrage vorbereitet, die alle Kunden abruft, deren Alter größer als der angegebene Parameterwert ist. Durch die Verwendung von **setInt()** wird der Wert für den Platzhalter in der Abfrage festgelegt.

Bei der Ausführung von SQL-Abfragen können verschiedene Arten von Fehlern auftreten, wie z.B. Verbindungsfehler, Syntaxfehler in der Abfrage oder Zugriffsverletzungen. Daher ist es wichtig, eine angemessene Fehlerbehandlung und Ausnahmeverarbeitung zu implementieren, um mit solchen Situationen umzugehen.

In den meisten Programmiersprachen gibt es Mechanismen zur Fehlerbehandlung und Ausnahmebehandlung. Verwenden Sie beispielsweise in Java die **try-catch**-Anweisungen, um potenzielle Ausnahmen abzufangen und entsprechend zu reagieren.

4.4 Objekt-Relationales Mapping (ORM)

ORM-Frameworks ermöglichen eine vereinfachte Datenbankprogrammierung, indem sie eine Abstraktionsebene zwischen relationalen Datenbanken und objektorientierten Programmiersprachen bieten. Mit ORM-Frameworks können Entwickler Datenbankoperationen mit objektorientierten Methoden durchführen, anstatt SQL-Abfragen direkt zu schreiben. Dies erleichtert die Modellierung, den Datenbankzugriff und die Durchführung von CRUD-Operationen. Beliebte ORM-Frameworks sind Hibernate für Java, Entity Framework für .NET und Django ORM für Python. Durch die Verwendung von ORM-Frameworks kann die Effizienz und Produktivität bei der Datenbankprogrammierung erhöht werden.

5. Weitere Datenzugriffsarten

5.1 Stored Procedures

Stored Procedures sind vordefinierte und gespeicherte SQL-Codeblöcke in einer Datenbank. Sie werden verwendet, um wiederholte Aufgaben oder komplexe Operationen auszuführen, indem sie eine Gruppe von SQL-Anweisungen in einer einzigen Einheit zusammenfassen. Stored Procedures werden auf dem Datenbankserver gespeichert und können von verschiedenen Anwendungen oder Benutzern aufgerufen werden.

Die Verwendung von Stored Procedures bietet mehrere Vorteile. Erstens können sie die Datenbankleistung verbessern, da sie einmal erstellt und kompiliert werden und dann bei Bedarf wiederverwendet werden können. Dies reduziert den Overhead bei der Übertragung von SQL-Code von der Anwendung zum Datenbankserver. Zweitens ermöglichen Stored Procedures eine bessere Sicherheit und Zugriffskontrolle, da sie anstelle von direkten Tabellenzugriffen verwendet werden können und Berechtigungen auf Prozedurebene vergeben werden können. Drittens fördern Stored Procedures die Datenkonsistenz, da sie

als Single-Point-of-Truth fungieren und sicherstellen können, dass bestimmte Operationen immer auf die gleiche Weise ausgeführt werden.

Um eine Stored Procedure zu erstellen, wird SQL-Code geschrieben, der die gewünschten Operationen enthält, und dieser Code wird dann mit einem eindeutigen Namen in der Datenbank gespeichert. Stored Procedures können Parameter akzeptieren, die es ermöglichen, Werte an die Prozedur zu übergeben und flexibelere Operationen durchzuführen. Um eine Stored Procedure auszuführen, wird sie einfach mit ihrem Namen aufgerufen und gegebenenfalls werden die erforderlichen Parameter übergeben.

Beispiel:

```
CREATE PROCEDURE GetCustomerByID
    @CustomerID INT
AS
BEGIN
    SELECT * FROM Customers WHERE CustomerID = @CustomerID
END
```

Erstellung einer Procedure zum Abruf aller Daten eines übergebenen Kunden(-ID)

In diesem Beispiel wird eine Stored Procedure mit dem Namen "GetCustomerByID" erstellt, die einen Parameter "@CustomerID" erwartet. Die Prozedur führt eine Abfrage aus, um den Kunden mit der angegebenen ID aus der Tabelle "Customers" abzurufen.

Die Verwendung von Stored Procedures ermöglicht eine bessere Modularität, Wiederverwendbarkeit und Sicherheit in der Datenbankprogrammierung. Sie sind besonders nützlich für komplexe Datenbankoperationen, die häufig wiederholt oder von verschiedenen Anwendungen aufgerufen werden müssen.

5.2 Views

Views sind virtuelle Tabellen in einer Datenbank, die aus einer oder mehreren vorhandenen Tabellen erstellt werden. Sie bieten eine logische Sicht auf die Daten, indem sie ausgewählte Spalten und Zeilen aus den zugrunde liegenden Tabellen abrufen und bestimmte Bedingungen oder Transformationen auf die Daten anwenden. Views werden verwendet, um den Datenzugriff zu vereinfachen und die Datenbankabfragen zu abstrahieren. Sie ermöglichen die Steuerung des Datenzugriffs, indem sie bestimmte Spalten ausblenden und Berechtigungen auf die View-Ebene festlegen. Darüber hinaus erleichtern Views komplexe Datenabfragen, indem sie vordefinierte SQL-Abfragen, Joins und Aggregationsfunktionen speichern. Sie verbessern die Datenkonsistenz und Datenintegrität, indem sie Daten aus verschiedenen Tabellen kombinieren oder transformieren und den Benutzern eine konsistente Sicht auf die Daten bieten. Views sind eine leistungsstarke Funktion in der Datenbankprogrammierung, um den Datenzugriff zu optimieren, die Datenverwaltung zu verbessern und die Entwicklungszeit zu verkürzen

```sql
CREATE VIEW CustomersWithHighOrders AS
SELECT CustomerName, OrderDate, TotalAmount
FROM Customers
JOIN Orders ON Customers.CustomerID = Orders.CustomerID
WHERE TotalAmount > 1000
```

Erstellung einer View mit Spalten aus der Tabelle Customer und Orders

In diesem Beispiel wird eine View mit dem Namen "CustomersWithHighOrders" erstellt. Diese View verbindet die Tabellen "Customers" und "Orders" und wählt nur die Kunden aus, deren Gesamtbestellwert größer als 1000 ist. Wenn die View erstellt wurde, kann sie wie eine normale Tabelle abgefragt werden, z.B. mit dem Befehl **SELECT * FROM CustomersWithHighOrders**.

Views bieten eine flexible Möglichkeit, den Datenzugriff zu optimieren, Datenzugriffssteuerung zu implementieren und die Datenabfrage zu vereinfachen. Sie sind ein leistungsstarkes Werkzeug in der Datenbankprogrammierung, um Datenlogik zu abstrahieren und die Datenverwaltung zu verbessern.

5.3 Indizes

Indizes sind Datenstrukturen in einer Datenbank, die verwendet werden, um den Zugriff auf Daten zu beschleunigen. Sie dienen dazu, bestimmte Spalten oder Kombinationen von Spalten in einer Tabelle zu organisieren, um das Auffinden von Daten basierend auf bestimmten Kriterien zu erleichtern. Indizes verbessern die Leistung von Datenbankabfragen, insbesondere bei großen Datenmengen.

Bei der Erstellung eines Indexes werden die Werte in den Indexspalten sortiert und in einer speziellen Datenstruktur gespeichert, die schnelles Suchen, Sortieren und Filtern ermöglicht. Wenn eine Abfrage ausgeführt wird, kann die Datenbank den Index nutzen, um schnell die relevanten Datensätze zu finden, anstatt die gesamte Tabelle durchsuchen zu müssen.

Die Verwendung von Indizes bietet mehrere Vorteile. Erstens beschleunigen sie den Lesezugriff auf Daten, da die Datenbank weniger Datenblöcke lesen muss, um die gewünschten Ergebnisse zu liefern. Zweitens verbessern Indizes die Leistung von Suchabfragen, Sortierungen und Verknüpfungen von Tabellen, da die benötigten Daten effizienter abgerufen werden können. Drittens ermöglichen Indizes eine bessere Datenintegrität, indem sie eindeutige Schlüssel oder Einschränkungen auf Spaltenwerte implementieren.

Es ist jedoch wichtig zu beachten, dass Indizes auch einige Nachteile haben. Indizes benötigen zusätzlichen Speicherplatz, da sie separate Datenstrukturen sind, die aktualisiert und gewartet werden müssen. Daher sollten Indizes sorgfältig ausgewählt und nur für Spalten erstellt werden, die häufig in Abfragen verwendet werden. Zu viele Indizes können

die Leistung von Schreibvorgängen beeinträchtigen, da die Datenbank die Indizes bei Aktualisierungen oder Einfügungen aktualisieren muss.

Die Verwendung von Indizes erfordert eine sorgfältige Planung und Analyse der Abfragemuster und Anforderungen der Anwendung. Durch das strategische Erstellen von Indizes können Sie die Leistung Ihrer Datenbankabfragen verbessern. Es ist wichtig, Indizes regelmäßig zu überprüfen und zu warten, um sicherzustellen, dass sie effizient bleiben und die Anforderungen der Anwendung erfüllen.

5.4 Trigger

Trigger sind Datenbankobjekte, die automatisch auf bestimmte Ereignisse in einer Datenbank reagieren und vordefinierte Aktionen ausführen. Sie werden verwendet, um komplexe Geschäftsregeln oder Datenintegritätsanforderungen zu implementieren und die Datenkonsistenz in einer Datenbank zu gewährleisten.

Ein Trigger besteht aus einem Ereignis, das auftritt, wie z.B. das Einfügen, Aktualisieren oder Löschen von Daten in einer Tabelle, und einer zugehörigen Aktion, die ausgeführt wird, wenn das Ereignis eintritt. Die Aktion kann eine einzelne SQL-Anweisung oder eine Gruppe von Anweisungen sein, die bestimmte Operationen auf den betroffenen Daten durchführen. Dies können beispielsweise das Aktualisieren von anderen Tabellen, das Generieren von Protokolleinträgen oder das Auslösen von Benachrichtigungen sein.

Die Verwendung von Triggern bietet mehrere Vorteile. Erstens ermöglichen sie die Automatisierung von Abläufen und die Durchsetzung von Geschäftsregeln auf Datenbankebene. Sie stellen sicher, dass bestimmte Operationen oder Bedingungen immer erfüllt werden, unabhängig von der verwendeten Anwendungslogik. Zweitens verbessern Triggern die Datenintegrität, indem sie vordefinierte Überprüfungen und Einschränkungen implementieren. Sie können beispielsweise sicherstellen, dass bestimmte Spaltenwerte nicht leer sind oder bestimmte Bedingungen erfüllen. Drittens bieten Triggern eine Möglichkeit zur Protokollierung von Datenänderungen und zur Verfolgung von Aktivitäten in der Datenbank.

Bei der Verwendung von Triggern ist es wichtig, die Ausführungsreihenfolge und die möglichen Auswirkungen auf die Leistung der Datenbank zu berücksichtigen. Falsch konfigurierte oder ineffizient geschriebene Trigger können die Datenbankleistung beeinträchtigen. Es ist auch wichtig, Triggern angemessene Sicherheitsberechtigungen zuzuweisen, um unbefugten Zugriff oder Missbrauch zu verhindern.

Triggern finden Anwendung in verschiedenen Szenarien, wie z.B. der Validierung von Daten, der Aktualisierung von Daten in verwandten Tabellen, der Erzeugung von Audit-Logs oder der Benachrichtigung bei bestimmten Ereignissen. Sie sind ein leistungsstarkes Werkzeug in der Datenbankprogrammierung, um komplexe Geschäftsregeln und Datenintegritätsanforderungen zu implementieren und die Datenkonsistenz zu gewährleisten.

5.5 Sicherheitsmechanismen

Die Benutzer- und Rollenverwaltung ist ein wichtiger Sicherheitsmechanismus in Datenbanksystemen. Sie ermöglicht die Zuweisung von Benutzerkonten und die Definition von Rollen mit spezifischen Zugriffsrechten. Durch die Vergabe individueller Benutzerkonten können Berechtigungen auf Datenbankebene gesteuert werden. Rollen wiederum ermöglichen eine vereinfachte Verwaltung von Zugriffsrechten, da sie einer Gruppe von Benutzern gemeinsame Berechtigungen zuweisen. Mit einer präzisen Benutzer- und Rollenverwaltung kann der Zugriff auf sensible Daten eingeschränkt und die Sicherheit der Datenbank erhöht werden.

Datenbanken bieten die Möglichkeit, Berechtigungen auf Tabellen- und Spaltenebene festzulegen. Dies ermöglicht eine granulare Steuerung des Zugriffs auf bestimmte Daten. Durch die Zuweisung von Berechtigungen wie SELECT, INSERT, UPDATE oder DELETE auf Tabellen und Spalten können bestimmte Operationen auf Daten eingeschränkt oder erlaubt werden. Dies ist besonders wichtig, um sicherzustellen, dass Benutzer nur auf die Daten zugreifen und diese ändern können, für die sie autorisiert sind. Die richtige Konfiguration von Berechtigungen trägt zur Datenintegrität und zum Schutz vor unbefugtem Zugriff bei.

Die Verschlüsselung von Daten ist ein wichtiger Sicherheitsmechanismus, um sensible Informationen in der Datenbank zu schützen. Durch die Verwendung von Verschlüsselungsalgorithmen werden Daten in der Datenbank verschlüsselt gespeichert und können nur mit dem entsprechenden Entschlüsselungsschlüssel gelesen werden. Dies gewährleistet, dass selbst bei einem unautorisierten Zugriff auf die Datenbank die Informationen unlesbar bleiben. Die Verschlüsselung kann auf Spaltenebene angewendet werden, um spezifische Daten zu schützen, oder auf der gesamten Datenbank, um eine umfassende Sicherheit zu gewährleisten.

Überwachungsmechanismen sind entscheidend, um verdächtige Aktivitäten in der Datenbank zu erkennen und auf potenzielle Sicherheitsverletzungen zu reagieren. Datenbanken bieten verschiedene Überwachungs- und Auditing-Funktionen, die Ereignisse wie Anmeldeversuche, Datenänderungen oder Zugriffsversuche aufzeichnen. Durch die Implementierung von Überwachungsregeln und das regelmäßige Überprüfen von Audit-Logs können ungewöhnliche Aktivitäten identifiziert und angemessene Maßnahmen ergriffen werden, um die Sicherheit der Datenbank zu gewährleisten

6. Zusammenfassung

In dieser Vorlesung haben wir die Grundlagen der Datenbank-Zugriffe behandelt und verschiedene Aspekte der Datenbankprogrammierung untersucht. Wir haben gelernt, wie man Datenbanken mit SQL abfragt und wie man Datenbanken in der Programmierung effektiv nutzt. Zusätzlich haben wir weitere Datenzugriffsarten kennengelernt, darunter Stored Procedures, Views, Indizes, Trigger, Schedules und Sicherheitsmechanismen.

Quellen

https://www.ibm.com/docs/de/db2/11.1?topic=objects-triggers

https://www.ibm.com/docs/de/db2/11.1?topic=triggers-types

https://www.ibm.com/docs/de/db2/11.1?topic=objects-indexes

https://de.wikipedia.org/wiki/CRUD

https://de.wikipedia.org/wiki/ACID

https://guide.jtl-software.de/jtl-wawi/vorlagen/anwendungsbeispiele-direkter-datenbank-zugriff/

https://de.wikipedia.org/wiki/Open_Database_Connectivity

https://support.microsoft.com/de-de/office/verwalten-von-odbc-datenquellen-b19f856b-5b9b-48c9-8b93-07484bfab5a7#:~:text=Open%20Database%20Connectivity%20(ODBC)%20ist,Microsoft%20SQL%20Server%2C%20verwenden%20können.

https://www.mysqltutorial.org/mysql-views-tutorial.aspx

https://docs.oracle.com/javase/tutorial/jdbc/basics/prepared.html

https://www.mysqltutorial.org/mysql-stored-procedure-tutorial.aspx

https://dev.mysql.com/doc/refman/8.0/en/create-procedure.html

https://dev.mysql.com/doc/refman/8.0/en/create-index.html

https://dev.mysql.com/doc/refman/8.0/en/trigger-syntax.html

https://dev.mysql.com/doc/refman/8.0/en/events-privileges.html

https://learn.microsoft.com/de-de/sql/relational-databases/security/authentication-access/database-level-roles?view=sql-server-ver16

https://www.ibm.com/docs/de/db2/10.5?topic=security-roles